WISE ROUND 3: SCENARIOS 3, 4, 5 — DETAILED

Card Ordering | Bill Split | Dispute Resolution

Full VIPER architecture + code + talk track for each

SCENARIO 3: CARD ORDERING FLOW (★★★★)

Their Prompt:

> "Walk us through how you'd design the card ordering experience. Users can order physical or virtual cards."

Clarifying Questions:

> "Are there different card types — debit, prepaid, multi-currency?"
> "Can users have multiple cards simultaneously?"
> "Does physical card ordering require identity verification in some countries?"
> "Is virtual card instantly usable, or does it need activation?"
> "Should I consider Apple Wallet integration for virtual cards?"

User Journey:

```
VIRTUAL CARD (instant):
User opens app → Card tab → "Get a card" → Select Virtual
→ Review (free / small fee) → Confirm with Face ID
→ Card details shown instantly → "Add to Apple Wallet" → Done

PHYSICAL CARD (days):
User opens app → Card tab → "Get a card" → Select Physical
→ Enter delivery address (Dynamic Forms — varies by country)
→ Review (fee + delivery estimate) → Confirm with Face ID
→ Order confirmed → Track: Ordered → Printed → Shipped → Delivered
→ Activate card (enter last 4 digits or scan) → Ready to use
```

High-Level Architecture:

```
┌──────────────────────────────────────────────────────────────────────────┐
│  WISE BACKEND                                                            │
│                                                                          │
│  Card Service ──→ Card Printer ──→ Shipping Partner                      │
│      │                                                                   │
│      ├── POST /v3/spend/profiles/{id}/cards/order                        │
```

```
│     ├── GET  /v3/spend/profiles/{id}/cards/{cardId}/status   │
│     └── POST /v3/spend/profiles/{id}/cards/{cardId}/activate │
│                                                              │
│  Dynamic Forms Service                                       │
│     └── GET /v3/forms/card-order/{country}                   │
│        Returns: field definitions per country                │
│                                                              │
│  Push Notification Service                                   │
│     └── Status updates: ordered → printed → shipped → delivered │
└──────────────────────────────────────────────────────────────┘
```

Mobile Architecture — VIPER with Coordinator:

```
CardOrderCoordinator (manages multi-step flow)
│
├── Step 1: CardTypeModule (VIPER)
│   ├── View: Shows card options (virtual/physical, debit/prepaid)
│   ├── Presenter: Formats card options, handles selection
│   ├── Interactor: Fetches available card types for user's country
│   ├── Router: Delegates to Coordinator for next step
│   └── Entity: CardType, CardOption
│
├── Step 2: DeliveryAddressModule (VIPER) — physical only
│   ├── View: Renders Dynamic Form from backend
│   ├── Presenter: Validates fields, formats errors
│   ├── Interactor: Fetches form schema, validates address via API
│   ├── Router: Delegates to Coordinator
│   └── Entity: AddressForm, FormField, ValidationResult
│
├── Step 3: ReviewOrderModule (VIPER)
│   ├── View: Shows summary — card type, address, fee, delivery estimate
│   ├── Presenter: Formats all details for display
│   ├── Interactor: Calculates total fee, checks eligibility
│   ├── Router: Delegates to Coordinator
│   └── Entity: OrderSummary
│
├── Step 4: ConfirmationModule (VIPER)
│   ├── View: Success state with card details or tracking
│   ├── Presenter: Routes to Apple Wallet or tracking
│   ├── Interactor: Submits order, handles virtual card provisioning
│   ├── Router: Navigate to card detail or tracking screen
│   └── Entity: OrderResult, CardDetails
│
└── Coordinator manages:
    ├── Flow direction (next/back between steps)
    ├── Shared state across steps (selected type, address)
```

```
├── Cancellation (discard partial order)
└── Deep link entry (push notification → tracking)
```

Code — Key Components:

```swift
```

```swift
// ═══ ENTITIES ═══

enum CardType: String, Codable {
    case virtual, physical
}

enum CardVariant: String, Codable {
    case debit, prepaid
}

struct CardOrderRequest: Codable {
    let cardType: CardType
    let variant: CardVariant
    let deliveryAddress: DeliveryAddress?  // nil for virtual
    let idempotencyKey: String             // Prevent duplicate orders
}

struct DeliveryAddress: Codable {
    let line1: String
    let line2: String?
    let city: String
    let postalCode: String
    let country: String
    // Different countries need different fields — Dynamic Forms handles this
}

struct CardOrderResult: Codable {
    let cardId: String
    let cardType: CardType
    let status: CardOrderStatus
    let cardDetails: VirtualCardDetails?  // Only for virtual — instant
    let estimatedDelivery: Date?          // Only for physical
}

enum CardOrderStatus: String, Codable {
    case ordered, printing, shipped, delivered, activated, failed
}

struct VirtualCardDetails: Codable {
    let cardNumber: String      // Show in app
    let expiryMonth: Int
    let expiryYear: Int
    let cvv: String             // Show once, then require auth to view again
}

// ═══ DYNAMIC FORMS ═══
```

```swift
// Backend sends the form structure — mobile renders it

struct DynamicForm: Codable {
    let fields: [FormField]
}

struct FormField: Codable {
    let id: String
    let label: String
    let type: FieldType        // text, dropdown, phone, postal
    let required: Bool
    let validation: String?    // Regex pattern
    let options: [String]?     // For dropdowns
    let placeholder: String?
}

enum FieldType: String, Codable {
    case text, dropdown, phone, postalCode, country
}

// ═══ COORDINATOR ═══

protocol CardOrderCoordinatorDelegate: AnyObject {
    func cardOrderDidComplete(cardId: String)
    func cardOrderDidCancel()
}

class CardOrderCoordinator {
    weak var delegate: CardOrderCoordinatorDelegate?
    private var navigationController: UINavigationController

    // Shared state across steps
    private var selectedCardType: CardType?
    private var selectedVariant: CardVariant?
    private var deliveryAddress: DeliveryAddress?

    func start() {
        let cardTypeModule = CardTypeBuilder.build(delegate: self)
        navigationController.pushViewController(cardTypeModule, animated: true)
    }

    // Called by Step 1 Router
    func didSelectCardType(_ type: CardType, variant: CardVariant) {
        selectedCardType = type
        selectedVariant = variant

        if type == .virtual {
```

```
            // Skip delivery address — go straight to review
            showReview()
        } else {
            showDeliveryAddress()
        }
    }

    // Called by Step 2 Router
    func didEnterAddress(_ address: DeliveryAddress) {
        deliveryAddress = address
        showReview()
    }

    // Called by Step 3 Router
    func didConfirmOrder() {
        showConfirmation()
    }

    private func showDeliveryAddress() {
        let country = getUserCountry()
        let addressModule = DeliveryAddressBuilder.build(country: country, delegate: self)
        navigationController.pushViewController(addressModule, animated: true)
    }

    private func showReview() {
        let summary = OrderSummary(
            cardType: selectedCardType!,
            variant: selectedVariant!,
            address: deliveryAddress,
            fee: calculateFee(),
            estimatedDelivery: selectedCardType == .physical ? estimateDelivery() : nil
        )
        let reviewModule = ReviewOrderBuilder.build(summary: summary, delegate: self)
        navigationController.pushViewController(reviewModule, animated: true)
    }

    private func showConfirmation() {
        // Submit order
    }
}

// ═══ INTERACTOR — Order Submission ═══

class CardOrderInteractor: CardOrderInteractorProtocol {
    weak var output: CardOrderInteractorOutput?
    private let apiClient: SpendAPIClientProtocol
    private let biometricAuth: BiometricAuthProtocol
```

```
    private let walletManager: AppleWalletManagerProtocol

    func submitOrder(_ request: CardOrderRequest) {
        Task {
            // Step 1: Biometric auth
            guard try await biometricAuth.authenticate(
                reason: "Confirm card order"
            ) else {
                await MainActor.run { output?.didFailAuth() }
                return
            }

            // Step 2: Submit with idempotency key
            do {
                let result = try await apiClient.orderCard(request)

                // Step 3: If virtual — offer Apple Wallet
                if result.cardType == .virtual, let details = result.cardDetails {
                    let canAddToWallet = await walletManager.canAddCard()
                    await MainActor.run {
                        output?.didOrderSuccess(result, canAddToWallet: canAddToWallet)
                    }
                } else {
                    await MainActor.run {
                        output?.didOrderSuccess(result, canAddToWallet: false)
                    }
                }
            } catch {
                await MainActor.run { output?.didOrderFail(error) }
            }
        }
    }
}

// ═══ APPLE WALLET INTEGRATION ═══

import PassKit

protocol AppleWalletManagerProtocol {
    func canAddCard() async -> Bool
    func addCard(details: VirtualCardDetails) async throws
}

class AppleWalletManager: AppleWalletManagerProtocol {
    func canAddCard() async -> Bool {
        PKPassLibrary.isPassLibraryAvailable() &&
        PKAddPaymentPassViewController.canAddPaymentPass()
```

```
    }

    func addCard(details: VirtualCardDetails) async throws {
        // Use PKAddPaymentPassViewController
        // Wise backend provides the encrypted card data
        // PassKit handles provisioning to Apple Wallet
    }
}

// ═══ TRACKING — Physical Card ═══

class CardTrackingInteractor: CardTrackingInteractorProtocol {
    private let apiClient: SpendAPIClientProtocol
    private var pollingTask: Task<Void, Never>?

    func startTracking(cardId: String) {
        pollingTask = Task {
            while !Task.isCancelled {
                do {
                    let status = try await apiClient.getCardStatus(cardId)
                    await MainActor.run { output?.didUpdateStatus(status) }

                    if status == .delivered || status == .activated {
                        break  // Stop polling
                    }
                } catch {
                    // Silent retry — tracking is background
                }
                try? await Task.sleep(nanoseconds: 60_000_000_000) // Poll every 60s
            }
        }
    }

    func stopTracking() {
        pollingTask?.cancel()
    }
}

// ═══ PRESENTER — Tracking Display ═══

class CardTrackingPresenter {
    func formatTimeline(_ status: CardOrderStatus) -> [TrackingStep] {
        let steps: [(CardOrderStatus, String, String)] = [
            (.ordered,   "Order placed",    "Your card is being prepared"),
            (.printing,  "Card printed",    "Your card design is ready"),
            (.shipped,   "Shipped",         "On its way to you"),
            (.delivered, "Delivered",       "Check your mailbox!"),
```

```
        (.activated, "Activated",       "Ready to use everywhere"),
    ]

    return steps.map { stepStatus, title, subtitle in
        TrackingStep(
            title: title,
            subtitle: subtitle,
            isCompleted: stepStatus.rawValue <= status.rawValue,
            isCurrent: stepStatus == status
        )
    }
  }
}
```

What to SAY:

> **On Coordinator:** "I'm using a Coordinator pattern to manage the multi-step flow. Each step is an independent VIPER module — CardType selection, delivery address, review, confirmation. The Coordinator holds shared state and controls the flow. This means I can reorder steps or skip steps (skip address for virtual cards) without changing any individual module."

> **On Dynamic Forms:** "Wise uses server-driven UI. For delivery address, the backend sends the form structure per country — UK needs a postcode format, India needs a PIN code, Germany needs a different layout. The mobile just renders whatever the backend sends. This means adding a new country's address format is a backend config change, not an app update."

> **On Apple Wallet:** "For virtual cards, I'd use PassKit to add to Apple Wallet. This gives customers contactless payment immediately — no waiting for a physical card. The user taps 'Add to Wallet', Wise's backend provides encrypted card data via the In-App Provisioning API, and PassKit handles the rest."

> **On Idempotency:** "Card ordering MUST be idempotent. If the network drops after the user confirms, the app retries. Without an idempotency key, they'd get two cards. I include a UUID as the idempotency key in every order request."

> **On the hard part — Virtual vs Physical divergence:** "The flows diverge significantly after confirmation. Virtual: instant details, show card number, offer Apple Wallet. Physical: tracking flow, push notifications at each stage, activation step. The Coordinator handles this branching cleanly — after confirmation, it checks the card type and presents the appropriate next module."

Their Likely Follow-Up Improvements:

They Say	Your Answer
"Add card replacement flow"	"New Coordinator entry point — starts at confirmation of replacement reason, skips type selection. Reuse address and review modules. Ship replacement, deactivate old card."

They Say	Your Answer
"Add card customization (name, design)"	"New VIPER module between type selection and review. Backend provides available designs. Preview the card with user's name overlaid. Store selection in Coordinator state."
"Handle activation"	"Physical card: user enters last 4 digits or scans the card with camera (Vision framework). API confirms activation. Status changes to .activated. Enable spend controls."

SCENARIO 4: BILL SPLIT (★★★)

Their Prompt:

> "After a restaurant payment, the user can split the bill with friends. Design this feature."

Clarifying Questions:

> "Can users split with non-Wise users too?"
> "Is it only equal split, or custom amounts?"
> "Can the split be modified after sending? What if someone declines?"
> "Is there a time limit for people to pay their share?"
> "Can users split any transaction, or only certain categories like restaurants?"

User Journey:

```
User sees restaurant transaction → taps "Split this bill"
→ Add participants (Wise contacts + phone/email for non-Wise)
→ Choose split type (equal / custom / percentage)
→ Review amounts → Confirm → Requests sent

PARTICIPANT JOURNEY:
Wise user: Push notification → "Pay S$33.33 for dinner at XYZ" → Tap to pay
Non-Wise user: SMS/WhatsApp link → "You owe S$33.33" → Link to pay (Wise or card)

TRACKING:
Original payer sees: "2 of 4 friends have paid" → Progress bar
Push notification when each person pays
```

High-Level Architecture:

```
┌──────────────────────────────────────────────────────────────┐
│ WISE BACKEND                                                 │
│                                                              │
```

```
│ Bill Split Service                          │
│ ├── POST /v3/spend/splits        (create split)      │
│ ├── GET  /v3/spend/splits/{id}    (get split status)   │
│ ├── POST /v3/spend/splits/{id}/pay (participant pays)   │
│ └── PUT  /v3/spend/splits/{id}    (modify split)      │
│                                             │
│ Notification Service                        │
│ ├── Push to Wise users                          │
│ ├── SMS/WhatsApp to non-Wise users (with payment link)  │
│ └── Push when participant pays their share              │
│                                             │
│ Payment Service                             │
│ ├── Wise-to-Wise transfer (free, instant)               │
│ └── External payment via link (card processing fee)     │
└──────────────────────────────────────────────────────────────┘
```

Mobile Architecture — VIPER:

```
BillSplitCoordinator
│
├── Step 1: SelectTransactionModule
│   └── Show eligible transactions (restaurants, bars, etc.)
│   └── Or: launched from transaction detail → skip this step
│
├── Step 2: AddParticipantsModule
│   ├── View: Contact picker + manual entry
│   ├── Presenter: Validate contacts, check Wise users
│   ├── Interactor: Search Wise contacts, resolve phone/email
│   ├── Entity: Participant (isWiseUser, contact info)
│   └── Show who's on Wise (free) vs external (may have fee)
│
├── Step 3: SetAmountsModule
│   ├── View: Split type selector + amount editor per person
│   ├── Presenter: Calculate splits, format amounts
│   ├── Interactor: Split calculation + rounding logic
│   ├── Entity: SplitType, ParticipantShare
│   └── Shows total = original amount (validates no money lost)
│
├── Step 4: ReviewAndSendModule
│   ├── View: Summary of who owes what
│   ├── Presenter: Format final review
│   ├── Interactor: Submit split request
│   └── Confirm with biometric auth
│
└── Step 5: TrackingModule
    ├── View: Progress bar + list of who paid / pending
```

```
├── Presenter: Format status per participant
├── Interactor: Poll for updates + handle push
└── Entity: SplitStatus, PaymentStatus
```

Code — The Hard Parts:

```swift
```

```swift
// ═══ ENTITIES ═══

struct BillSplit: Codable, Identifiable {
    let id: String
    let originalTransaction: CardTransaction
    let participants: [Participant]
    let splitType: SplitType
    let status: SplitStatus
    let createdAt: Date
}

struct Participant: Codable, Identifiable {
    let id: String
    let name: String
    let contact: String             // Phone or email
    let isWiseUser: Bool
    let wiseProfileId: String?
    var amount: Decimal
    var paymentStatus: PaymentStatus
}

enum SplitType: Codable {
    case equal
    case custom
    case percentage([Decimal])      // e.g., [0.5, 0.3, 0.2]
}

enum PaymentStatus: String, Codable {
    case pending, paid, declined, expired
}

enum SplitStatus: String, Codable {
    case active, completed, cancelled
}

// ═══ THE ROUNDING PROBLEM (Critical for fintech!) ═══

class BillSplitCalculator {

    /// Split equally — last person absorbs rounding remainder
    /// RULE: Never create or lose money through rounding
    func splitEqually(amount: Decimal, participantCount: Int) -> [Decimal] {
        guard participantCount > 0 else { return [] }

        // Round each share to 2 decimal places
        let rawShare = amount / Decimal(participantCount)
```

```
    let roundedShare = roundToTwo(rawShare)

    // Calculate what N-1 people pay
    let othersTotal = roundedShare * Decimal(participantCount - 1)

    // Last person gets the remainder — ensures exact total
    let lastPersonShare = amount - othersTotal

    var shares = Array(repeating: roundedShare, count: participantCount - 1)
    shares.append(lastPersonShare)

    return shares

    // Example: S$100 ÷ 3
    // rawShare = 33.333...
    // roundedShare = 33.33
    // othersTotal = 33.33 × 2 = 66.66
    // lastPersonShare = 100 - 66.66 = 33.34
    // Result: [33.33, 33.33, 33.34] → total = 100.00 ✅
}

/// Split by percentage
func splitByPercentage(amount: Decimal, percentages: [Decimal]) -> [Decimal] {
    guard !percentages.isEmpty else { return [] }
    // Validate percentages sum to 1.0 (or 100%)
    let total = percentages.reduce(Decimal.zero, +)
    guard total == Decimal(1) else { return [] }  // Must sum to 100%

    var shares = percentages.map { roundToTwo(amount * $0) }

    // Adjust last share for rounding
    let sharesTotal = shares.dropLast().reduce(Decimal.zero, +)
    shares[shares.count - 1] = amount - sharesTotal

    return shares
}

/// Custom amounts — just validate they sum to total
func validateCustomSplit(amount: Decimal, shares: [Decimal]) -> Bool {
    let total = shares.reduce(Decimal.zero, +)
    return total == amount  // Must match exactly
}

private func roundToTwo(_ value: Decimal) -> Decimal {
    var result = value
    var rounded = Decimal()
    NSDecimalRound(&rounded, &result, 2, .bankers)  // Banker's rounding
```

```
        return rounded
    }
}

// ═══ INTERACTOR — Split Creation ═══

class BillSplitInteractor: BillSplitInteractorProtocol {
    weak var output: BillSplitInteractorOutput?
    private let apiClient: SpendAPIClientProtocol
    private let contactResolver: ContactResolverProtocol
    private let calculator = BillSplitCalculator()

    func resolveContacts(_ contacts: [String]) {
        Task {
            var participants: [Participant] = []

            for contact in contacts {
                // Check if they're a Wise user
                let wiseProfile = try? await apiClient.lookupWiseUser(contact: contact)

                participants.append(Participant(
                    id: UUID().uuidString,
                    name: wiseProfile?.name ?? contact,
                    contact: contact,
                    isWiseUser: wiseProfile != nil,
                    wiseProfileId: wiseProfile?.id,
                    amount: 0,  // Set later
                    paymentStatus: .pending
                ))
            }

            await MainActor.run {
                output?.didResolveContacts(participants)
            }
        }
    }

    func calculateSplit(amount: Decimal, type: SplitType, participantCount: Int) {
        let shares: [Decimal]

        switch type {
        case .equal:
            shares = calculator.splitEqually(amount: amount, participantCount: participantCount)
        case .percentage(let pcts):
            shares = calculator.splitByPercentage(amount: amount, percentages: pcts)
        case .custom:
            shares = []  // User enters manually
```

```
        }

        output?.didCalculateShares(shares)
    }

    func submitSplit(transaction: CardTransaction, participants: [Participant]) {
        Task {
            do {
                let split = try await apiClient.createSplit(
                    transactionId: transaction.id,
                    participants: participants
                )

                // Backend handles:
                // - Wise users: push notification with payment request
                // - Non-Wise: SMS/WhatsApp with payment link

                await MainActor.run { output?.didCreateSplit(split) }
            } catch {
                await MainActor.run { output?.didFailCreateSplit(error) }
            }
        }
    }
}

// ═══ PRESENTER — Amount Formatting ═══

class BillSplitPresenter: BillSplitPresenterProtocol {
    weak var view: BillSplitViewProtocol?
    var interactor: BillSplitInteractorProtocol
    var router: BillSplitRouterProtocol

    private let formatter = CurrencyFormatter()

    func didUpdateParticipants(_ participants: [Participant]) {
        let viewModels = participants.map { p in
            ParticipantCellViewModel(
                name: p.name,
                formattedAmount: formatter.format(p.amount, currency: "SGD"),
                isWiseUser: p.isWiseUser,
                statusLabel: p.isWiseUser ? "Free & instant" : "Via payment link",
                statusIcon: p.isWiseUser ? "checkmark.circle.fill" : "link"
            )
        }
        view?.showParticipants(viewModels)
    }
```

```
    func didCalculateShares(_ shares: [Decimal], total: Decimal) {
        let formattedShares = shares.map { formatter.format($0, currency: "SGD") }
        let sumCheck = shares.reduce(Decimal.zero, +)

        // Always show total validation to user
        view?.showShares(
            formattedShares,
            totalLabel: "Total: \(formatter.format(total, currency: "SGD"))",
            isBalanced: sumCheck == total  // Must be true!
        )
    }
}

// ═══ TRACKING ═══

class SplitTrackingInteractor {
    func startTracking(splitId: String) {
        // Listen for push notifications when participants pay
        // Update progress: "2 of 4 friends have paid"
        // When all paid → status = .completed
    }
}
```

What to SAY:

> **On rounding:** "The most critical fintech detail here. S$100 split 3 ways — naive division gives 33.333 repeating. If I round each to 33.33, the total is 99.99 — we've lost a cent. My approach: N-1 people pay the rounded amount, the last person pays the remainder. Total is always exactly the original amount. No money created, no money lost."

> **On Wise vs non-Wise:** "Two very different flows. Wise users get a push notification and can pay with one tap — free and instant, it's just a balance transfer. Non-Wise users get an SMS or WhatsApp link leading to a payment page where they can pay by card. This has processing fees, which the Presenter should display upfront so the user knows."

> **On Banker's rounding:** "I use NSDecimalRound with .bankers rounding — also called 'round half to even.' This reduces systematic rounding bias over many transactions. Standard practice in financial software."

> **On the Coordinator:** "The bill split flow has conditional steps — if launched from a transaction detail, skip the transaction selection step. The Coordinator manages this branching without each module knowing about the other steps."

Their Likely Follow-Up:

They Say	Your Answer
"What if someone declines?"	"The split remains active. Show declined status on that participant. Original payer can: resend request, remove them and redistribute, or absorb their share. Backend recalculates amounts if participants change."
"Add expense tracking"	"New VIPER module — SpendingSummaryModule. Aggregate splits by month, show total recovered, outstanding debts. Backend provides summary API, mobile just displays."
"What about different currencies?"	"If I split with someone whose balance is in a different currency, we need FX conversion. Show both amounts: 'John owes €15.00 (S$22.50 from your SGD)'. Rate locked at split creation time, not payment time — otherwise amounts keep changing."

SCENARIO 5: DISPUTE / CHARGEBACK FLOW (★★★)

Their Prompt:

> "A customer doesn't recognize a transaction or has an issue with a purchase. Design the dispute flow."

Clarifying Questions:

> "What dispute reasons should we support?"
> "Can users upload evidence like receipts or screenshots?"
> "Is there a provisional credit while the dispute is being investigated?"
> "What's the regulatory timeline — how long does the user have to dispute?"
> "Should we handle merchant disputes differently from fraud claims?"

User Journey:

```
FILING A DISPUTE:
User sees suspicious transaction → taps → "Something wrong?"
→ Select reason (don't recognize / wrong amount / duplicate / not received / fraud)
→ Provide details (text description + upload evidence photos)
→ Review claim summary
→ Confirm with biometric auth
→ "Dispute filed. We'll investigate within 10 business days."
→ If eligible: provisional credit applied to balance

TRACKING RESOLUTION:
Push notification: "We're reviewing your dispute"
```

```
Push notification: "We need more information" → deep link to provide
Push notification: "Dispute resolved in your favor — S$45.00 refunded"
OR: "Dispute denied — provisional credit reversed"

TIMELINE:
Day 0: Filed
Day 1-3: Under review (initial investigation)
Day 3-10: With card network (Visa/Mastercard investigation)
Day 10-45: Resolution
Day 45-120: Escalation if customer disagrees
```

High-Level Architecture:

```
┌──────────────────────────────────────────────────────────────────────┐
│ WISE BACKEND                                                         │
│                                                                      │
│ Dispute Service                                                      │
│ ├── POST /v3/spend/disputes            (file dispute)                │
│ ├── GET  /v3/spend/disputes/{id}       (get status)                  │
│ ├── POST /v3/spend/disputes/{id}/evidence (upload files)             │
│ ├── PUT  /v3/spend/disputes/{id}       (update info)                 │
│ └── GET  /v3/spend/disputes            (list all)                    │
│                                                                      │
│ Chargeback Service (talks to Visa/Mastercard)                        │
│ ├── Submit chargeback to card network                                │
│ ├── Receive network decision                                         │
│ └── Process provisional credit / reversal                            │
│                                                                      │
│ File Storage Service (S3)                                            │
│ └── Encrypted storage for evidence photos/receipts                   │
│                                                                      │
│ Push Notification Service                                            │
│ └── Status updates at each stage                                     │
└──────────────────────────────────────────────────────────────────────┘
```

Mobile Architecture — VIPER with Coordinator:

```
DisputeFlowCoordinator
│
├── Step 1: SelectReasonModule (VIPER)
│   ├── View: List of dispute reasons with icons
│   ├── Presenter: Shows reasons, validates eligibility
│   ├── Interactor: Check if transaction is disputable
│   │   (within timeframe? not already disputed?)
│   ├── Router: Delegates to Coordinator
```

```
│   └── Entity: DisputeReason enum
│
├── Step 2: ProvideDetailsModule (VIPER)
│   ├── View: Text field + photo upload area
│   ├── Presenter: Validates input, shows upload progress
│   ├── Interactor: Compress images, upload to backend
│   ├── Router: Delegates to Coordinator
│   └── Entity: DisputeEvidence
│
├── Step 3: ReviewDisputeModule (VIPER)
│   ├── View: Summary — transaction, reason, evidence, timeline
│   ├── Presenter: Format review, show provisional credit info
│   ├── Interactor: Submit dispute + biometric auth
│   ├── Router: Navigate to tracking
│   └── Entity: DisputeSubmission
│
└── Step 4: DisputeTrackingModule (VIPER)
    ├── View: Timeline visualization + status + actions needed
    ├── Presenter: Format timeline steps, show deadlines
    ├── Interactor: Poll status, handle push updates
    ├── Router: Deep link from push → specific dispute
    └── Entity: DisputeStatus, DisputeTimeline
```

Code — Key Components:

```swift
```

```
// ═══ ENTITIES ═══

enum DisputeReason: String, Codable, CaseIterable {
    case unrecognized       // "I don't recognize this transaction"
    case wrongAmount        // "I was charged the wrong amount"
    case duplicate          // "I was charged twice"
    case notReceived        // "Item/service not received"
    case defective          // "Item was defective or not as described"
    case cancelledService   // "I cancelled but was still charged"
    case fraud              // "My card was used without my permission"

    var title: String {
        switch self {
        case .unrecognized: return "I don't recognize this"
        case .wrongAmount: return "Wrong amount charged"
        case .duplicate: return "Charged twice"
        case .notReceived: return "Never received item/service"
        case .defective: return "Item not as described"
        case .cancelledService: return "Cancelled but still charged"
        case .fraud: return "Unauthorized use of my card"
        }
    }

    var icon: String {
        switch self {
        case .unrecognized: return "questionmark.circle"
        case .wrongAmount: return "dollarsign.circle"
        case .duplicate: return "doc.on.doc"
        case .notReceived: return "shippingbox"
        case .defective: return "exclamationmark.triangle"
        case .cancelledService: return "xmark.circle"
        case .fraud: return "lock.shield"
        }
    }

    /// Fraud disputes get expedited handling + immediate card freeze suggestion
    var isFraud: Bool { self == .fraud || self == .unrecognized }

    /// How much evidence is needed
    var requiresEvidence: Bool {
        switch self {
        case .wrongAmount, .notReceived, .defective, .cancelledService: return true
        case .unrecognized, .duplicate, .fraud: return false  // Wise investigates
        }
    }
}
```

```swift
struct DisputeEvidence: Codable {
    let description: String
    let photos: [EvidencePhoto]
    let expectedAmount: Decimal?    // For wrongAmount disputes
}

struct EvidencePhoto: Codable {
    let id: String
    let uploadURL: String
    let thumbnailURL: String
    let uploadStatus: UploadStatus
}

enum UploadStatus: String, Codable {
    case uploading, completed, failed
}

struct Dispute: Codable, Identifiable {
    let id: String
    let transaction: CardTransaction
    let reason: DisputeReason
    let evidence: DisputeEvidence?
    let status: DisputeStatus
    let provisionalCredit: Decimal?
    let filedAt: Date
    let deadlineAt: Date           // Regulatory deadline
    let timeline: [DisputeTimelineEvent]
}

enum DisputeStatus: String, Codable {
    case filed
    case underReview
    case moreInfoNeeded
    case withCardNetwork
    case resolvedInFavor      // Customer wins — refund
    case resolvedAgainst      // Customer loses — provisional credit reversed
    case escalated
}

struct DisputeTimelineEvent: Codable {
    let status: DisputeStatus
    let timestamp: Date
    let message: String
}

// ═══ INTERACTOR — Eligibility Check ═══
```

```swift
class DisputeEligibilityInteractor {

    func checkEligibility(transaction: CardTransaction) -> DisputeEligibilityResult {
        // Rule 1: Transaction must be completed (not pending)
        guard transaction.status == .completed else {
            return .ineligible("Transaction is still processing. Wait for it to complete.")
        }

        // Rule 2: Must be within dispute window (120 days for Visa)
        let daysSinceTransaction = Calendar.current.dateComponents(
            [.day], from: transaction.timestamp, to: Date()
        ).day ?? 0

        guard daysSinceTransaction <= 120 else {
            return .ineligible("This transaction is over 120 days old and can no longer be disputed.")
        }

        // Rule 3: Not already disputed
        // Check with backend

        // Rule 4: Reversed transactions can't be disputed
        guard transaction.status != .reversed else {
            return .ineligible("This transaction has already been refunded.")
        }

        return .eligible(daysRemaining: 120 - daysSinceTransaction)
    }
}

enum DisputeEligibilityResult {
    case eligible(daysRemaining: Int)
    case ineligible(String)
}

// ═══ INTERACTOR — Evidence Upload ═══

class DisputeEvidenceInteractor: DisputeEvidenceInteractorProtocol {
    weak var output: DisputeEvidenceInteractorOutput?
    private let apiClient: SpendAPIClientProtocol
    private let imageCompressor: ImageCompressorProtocol

    func uploadEvidence(photos: [UIImage], disputeId: String) {
        Task {
            var uploaded: [EvidencePhoto] = []

            for (index, photo) in photos.enumerated() {
```

```
            do {
                // Step 1: Compress image (max 2MB)
                let compressed = try imageCompressor.compress(
                    photo, maxSizeKB: 2048, quality: 0.7
                )

                // Step 2: Get upload URL from backend
                let uploadURL = try await apiClient.getEvidenceUploadURL(
                    disputeId: disputeId
                )

                // Step 3: Upload with progress
                let result = try await apiClient.uploadFile(
                    data: compressed,
                    to: uploadURL,
                    onProgress: { progress in
                        Task { @MainActor in
                            self.output?.didUpdateUploadProgress(
                                index: index,
                                progress: progress
                            )
                        }
                    }
                )

                uploaded.append(result)

            } catch {
                await MainActor.run {
                    output?.didFailUpload(index: index, error: error)
                }
            }
        }

        await MainActor.run {
            output?.didCompleteUploads(uploaded)
        }
    }
  }
}

// ═══ PRESENTER — Timeline Formatting ═══

class DisputeTrackingPresenter: DisputeTrackingPresenterProtocol {
  weak var view: DisputeTrackingViewProtocol?

  func formatDispute(_ dispute: Dispute) {
```

```
    // Timeline steps
    let timelineSteps = formatTimeline(dispute)

    // Deadline warning
    let daysRemaining = Calendar.current.dateComponents(
        [.day], from: Date(), to: dispute.deadlineAt
    ).day ?? 0

    let deadlineLabel: String
    let deadlineUrgent: Bool
    if daysRemaining <= 7 {
        deadlineLabel = "⚠️ \(daysRemaining) days remaining to provide information"
        deadlineUrgent = true
    } else {
        deadlineLabel = "\(daysRemaining) days remaining"
        deadlineUrgent = false
    }

    // Provisional credit info
    let creditLabel = dispute.provisionalCredit.map { credit in
        "Provisional credit of \(formatCurrency(credit)) applied to your balance"
    }

    // Action needed?
    let actionNeeded = dispute.status == .moreInfoNeeded
    let actionLabel = actionNeeded ? "We need more information from you" : nil

    view?.showDisputeDetail(
        timeline: timelineSteps,
        deadlineLabel: deadlineLabel,
        deadlineUrgent: deadlineUrgent,
        creditLabel: creditLabel,
        actionNeeded: actionNeeded,
        actionLabel: actionLabel
    )
}

private func formatTimeline(_ dispute: Dispute) -> [TimelineStepViewModel] {
    let allSteps: [(DisputeStatus, String)] = [
        (.filed, "Dispute filed"),
        (.underReview, "Under review"),
        (.withCardNetwork, "Investigating with card network"),
        (.resolvedInFavor, "Resolved"),
    ]

    return allSteps.map { status, label in
        let event = dispute.timeline.first { $0.status == status }
```

```
        return TimelineStepViewModel(
            label: label,
            date: event?.timestamp,
            message: event?.message,
            isCompleted: event != nil,
            isCurrent: dispute.status == status
        )
    }
  }
}

// ═══ FRAUD SPECIAL HANDLING ═══

class FraudDisputeInteractor {
  func handleFraudReport(transaction: CardTransaction) {
    // 1. Immediately suggest freezing the card
    output?.suggestCardFreeze(
        message: "We recommend freezing your card immediately to prevent further unauthorized use."
    )

    // 2. File dispute with expedited flag
    // Fraud disputes are prioritized by the backend

    // 3. If multiple unrecognized transactions, flag for bulk dispute
    // "We found 3 other transactions from the same merchant. Include them?"
  }
}
```

What to SAY:

> **On eligibility:** "Before showing the dispute form, I check eligibility — is the transaction completed? Within 120 days? Not already disputed? This prevents wasted effort and reduces backend load from invalid submissions."

> **On evidence upload:** "I compress images before upload — phones take 12MP photos but we don't need that resolution for receipts. Max 2MB, quality 0.7. Upload with progress indicator so the user knows it's working. Resumable uploads in case network drops."

> **On fraud:** "Fraud disputes get special treatment. Immediately suggest freezing the card — don't wait for the dispute process. If we find multiple suspicious transactions from the same merchant or location, offer to bulk-dispute them."

> **On provisional credit:** "Wise may apply provisional credit to the balance during investigation. The tricky mobile UX is showing this clearly — the balance went UP, but it might go DOWN again if the dispute is denied. I'd show provisional amounts in a different color with a label: 'Provisional — pending dispute resolution.'"

> **On deadlines:** "Card networks have regulatory timelines. The mobile app must show days remaining prominently, and push a notification when the deadline is approaching. If the user needs to provide more info and doesn't, the dispute auto-closes — that's money lost."

> **On the hard part — file upload:** "Image upload is the trickiest mobile concern. Large files over cellular network, progress indication, retry on failure, compression to reduce size, and the privacy concern — evidence photos should be encrypted in transit and at rest. I'd use a background URLSession so uploads continue even if the user leaves the screen."

Their Likely Follow-Up:

They Say	Your Answer
"What about dispute for a pending transaction?"	"Show 'Wait until transaction completes' message. Pending = authorization hold, not settled. The amount might change (restaurant tip). Once settled, enable dispute."
"How to handle provisional credit reversal?"	"Push notification: 'Dispute resolved — provisional credit of S$45 reversed.' Show in transaction list as a new line item. Balance updates accordingly. Red warning color, not the usual green."
"Add merchant communication"	"New VIPER module — MerchantChatModule. Before filing formal dispute, offer 'Contact merchant first.' Some issues (wrong amount, not received) are faster to resolve directly. Only escalate to formal dispute if merchant doesn't respond in 7 days."
"Offline filing"	"Allow user to fill the form offline and queue the submission. Store locally in encrypted CoreData. Upload evidence when online. Show 'Pending submission' status. Auto-submit when network available."

COMPARISON TABLE — ALL 5 SCENARIOS

Aspect	Card Transactions	3DS	Card Ordering	Bill Split	Disputes
Hard part	Real-time push + multi-currency	5-min expiry + biometric	Virtual vs physical flow	Rounding + Wise vs non-Wise	File upload + regulatory timelines
VIPER modules	1 (list + detail)	1 (approval screen)	4 (Coordinator)	5 (Coordinator)	4 (Coordinator)
Backend events	Transaction webhook	3DS challenge webhook	Status updates	Payment received	Status changes
Push notifications	Instant per transaction	Time-critical challenge	Status per stage	Per participant payment	Per status change

Aspect	Card Transactions	3DS	Card Ordering	Bill Split	Disputes
Biometric auth	No (just viewing)	Yes (before approve)	Yes (before order)	Yes (before send)	Yes (before submit)
Offline	Show cached list	No (time-critical)	Save draft address	No (needs network)	Queue submission
Your BFX connection	FX conversion logic	Auth flows	Country rollouts	Multi-payment	Buyer protection